2013 17th International Conference on Information Visualisation

Interactive Curriculum Visualization

Harri Siirtola, Kari-Jouko Räihä, and Veikko Surakka
Tampere Unit for Computer-Human Interaction (TAUCHI)
School of Information Sciences,University of Tampere, Finland
{*harri.siirtola, kari-jouko.raiha, veikko.surakka*}*@uta.fi*

***Abstract*—In curriculum visualization, information visualization techniques are used to communicate the structure and content of a curriculum to the stakeholders, such as students, lecturers, and administrators. Curriculum visualization has recently drawn a lot of interest as there are growing demands to form larger educational units and to find savings by eliminating overlap. We have developed a novel approach to analyse and visualize the contents of a curriculum. Our software tool for curriculum visualization uses a fast heuristic to automatically lay out a curriculum diagram, provides tools to process the content information, and offers several coordinated views to visualize the curriculum contents and overlap. The approach and software tool have already been used successfully in one survey and two others are underway.**

***Keywords*-curriculum visualization**

I. Introduction

Educational institutions publish their course information as course catalogues, usually both in print and on the web. These mainly textual descriptions are good for conveying detailed information about individual courses. Often descriptions are accompanied with node-link diagrams that describe the preferred or requisite sequence in which the courses should be taken, such as in Fig. 1.

This kind of curriculum description is mainly targeted at the current and prospective students of the institution, but there are other stakeholders as well. Lecturers, who constantly update and improve their courses, often need to know whether a certain topic is covered elsewhere in the curriculum, and the textual course descriptions can be too vague to answer this. Management of the institution might be interested in exploring if there is any overlap in the curriculum, and thereby possibilities to optimise the use of resources. If we need to understand the overlap in curriculums between different institutions, these problems are even harder because of the often non-compatible course descriptions.

Curriculum visualization is becoming increasingly important as our education systems are faced with demands of merging into larger units, such as university alliances and consortiums. These mergers include difficult decisions about eliminating overlap in degree program contents to cut costs. Decision-makers need to understand the extent and nature of the overlap to make informed decisions, often without in-depth knowledge about specific disciplines.

The information visualization approach [1], [2] and associated techniques can make these tasks easier. For this purpose we developed a novel approach to explore and visualize a curriculum and a computer program to interact with such visualizations. This paper describes the visualization method and our interactive tool for analysing curriculum data, and reports our experiences from a project where two units providing extensive education in HCI (Human-Computer Interaction) used the tool to analyse their curriculums for overlap (Tampere Unit for Computer-Human Interaction (TAUCHI) from the University of Tampere and Human-Centered Technology (IHTE) from Tampere University of Technology).

The structure of this paper is as follows. After reviewing related work, the process of acquiring detailed course diagrams is explained. Then the representations for different aspects of curriculum data are described, followed by an account of how the interaction with the visualization is implemented. Finally, we report our experiences from using this approach and the software application, and present our conclusions.

II. Related work

Curriculum visualization is related to *curriculum mapping* [3], a process where an operational curriculum is reviewed. Curriculum mapping is defined by Hege et al. [4] as "a reality-based record of the content actually taught, how long it was taught, and the match between that was taught and what was assessed".

Descriptions of curriculums are text, and many of the curriculum visualization techniques are inspired by general text visualization methods (see Card et al. [5], Ch. 5) for a review). Often this means treating the text as high-dimensional data (i.e., vectors of word frequencies) and then using multidimensional scaling (MDS, see [6] for methods) to 'flatten' these high-dimensional values into 2D or 3D entities.

Johnson et al. [7] present a system that shares some goals with ours: it is aimed at discovering overlapping content of courses, and it makes use of interaction for exploring the data. However, our system scales up to large data collections, allowing comparisons of full curriculums, not just courses. In addition, we use interaction for exploring the visualization

1550-6037/13 $26.00 © 2013 IEEE
DOI 10.1109/IV.2013.13

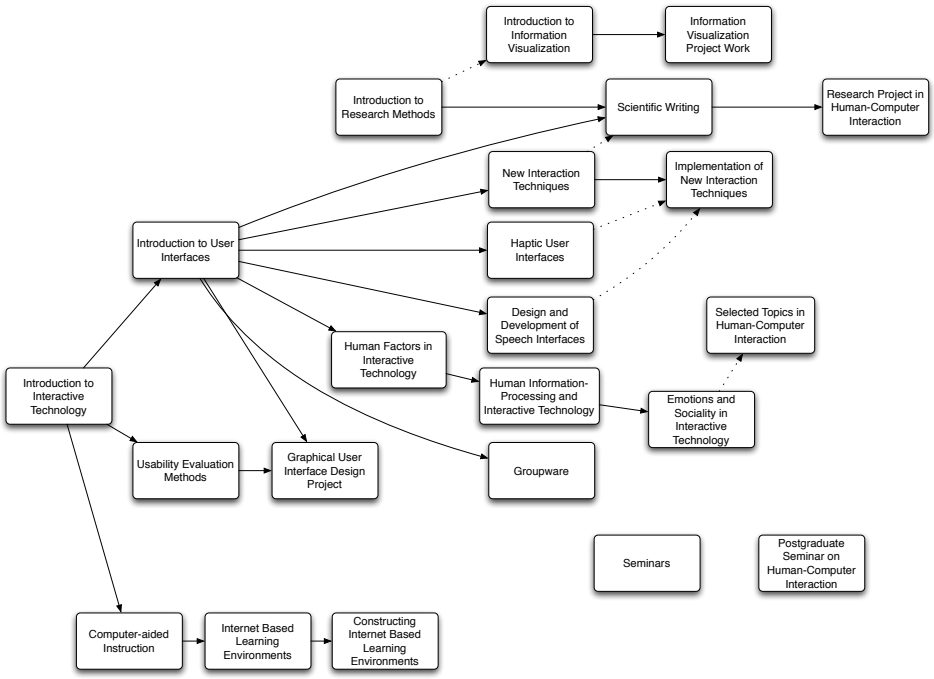

Figure 1. Typical curriculum diagram: courses and their recommended (dashed line) or requisite (solid line) sequence.

itself, not the underlying relational database, as Johnson et al. in effect do.

Sant [8] has implemented a curriculum visualization system that is based on data mining and MDS. In his system, course descriptions are edited to contain only the knowledge learning outcomes, skill learning outcomes, and a topical outline. Then vectors of word frequencies are constructed and a 2D map visualization of this multidimensional data structure is produced. This method requires very little preprocessing of course outlines, but the rendering is naturally dependent on the accuracy and detail in the syllabi.

CurricViz [9] is a prototype curriculum visualization system based on a Prolog [10] description of the curriculum. The system automatically creates a node-link diagram akin to figure 1 that illustrates the curriculum structure and prerequisite-or-parallel relationships. However, CurricViz does not currently depict or analyse the course content in any way.

Sommaruga and Catenazzi [11] have experimented with representing an education programme as a 3D virtual environment. Their implementation creates a 3D landscape where all the curriculum data (descriptions, credits, and durations) are translated into graphical form. They claim that this approach creates a simple, effective, and intuitive interface to a curriculum, but they do not have empirical evidence of this.

Prasolova-Førland [12] reports experiences of using a 3D Collaborative Virtual Environment (CVE) as a tool to visualize curriculum content. She represents curriculum data as virtual spaces where the users can move and observe objects (e.g., signs and images). Her emphasis is more on the collaborative activity of designing 3D curriculum visualizations than the visualization itself.

Another important related problem is ontology visualization which resembles curriculum visualization a great deal. An *ontology* is a set of concepts and their interrelations in some domain. Ontologies have many uses, such as improving information search and retrieval in the semantic web, digital libraries, and personal information management. The notion of ontology visualization has strong parallels with curriculum visualization, as the 'sets of concepts and their interrelations' can be seen as 'sets of courses and concepts in a curriculum and their interrelations'. Katifori et al. [13], [14] made a survey of proposed ontology visualization methods and recognised the following classes:

- Indented lists
- Node-link diagrams (i.e., trees and networks)
- Zoomable user interfaces (ZUIs)
- Space-filling methods (e.g. Treemaps)
- Distortion-oriented techniques (e.g. Focus+Context, Overview+Detail)
- 3D information landscapes

They conclude that there does not seem to be a single visualization method that would be appropriate for all applications, and that there should be several, complementary views according to users' current needs. In addition, they note that visualizations should be coupled with effective search or query tools.

Pretorius [15] has implemented an ontology base visualization tool called *LexoVis*, which combines a number of approaches, such as clustering and ordering heuristics, to give insight into large ontology bases. Ontology engineers have found his tool useful in analysing sets of lexons (a *lexon* is a conceptual construct that depicts a generic semantic relationship in some domain).

The Ontology of Computing project [16] is developing an ontology of all of computing, with support from NSF, ACM Education Board, and IEEE-CS Education Activities Board. The working group has considered how to visualize the structure of over 2000 nodes and even more links, and their proposition is a node-link diagram that will present one part of the hierarchy at a time, with hyperlinks to move around.

Kriglstein [17] presents in [18] a structure of an ontology for computer science curriculum and compares three visual representations for it. She concludes that there is no single visualization technique that would fulfil all the requirements, so we need to combine different techniques depending on the users' tasks.

III. Acquiring curriculum data

There are often problems in textual course descriptions, such as being out of date, being too vague, or being prepared for marketing rather than informative purposes. There are fairly good reasons for these issues (e.g. constantly changing lecturers and rapidly evolving topics), but they make it hard to compare the substance of courses. Because of these issues, we decided not to utilise the course descriptions as such.

We propose an alternative method to gain more detailed insight into actual substance of the courses, and thus allowing a comparison of their contents. In the following, we describe how we abstracted and extracted the course contents.

A. Abstraction for curriculum data

It is common to apply a top-down approach in visualizing curriculum content. In this approach one lists the courses, corresponding learning objectives, and course outline at a relatively high level. Instead of this, we chose to construct the curriculum visualization by using a bottom-up approach where the contents of a course was synthesised from the topics that the course covers. A *topic* is an atomic unit of teaching we make use of. If the course is about data structures, then some of the typical topics would be abstract data types 'list', 'stack', 'heap', and so on. Topics may overlap or even subsume each other. Continuing the data structures example, 'abstract data structure' could be a topic as well, although the earlier topics might fall within it.

Lecturers were instructed to describe the contents of their courses by listing every topic they discuss within the course, and to assign a weight to each topic on a scale from 1 to 5. The scale represented the degree to which the topic was dealt with during the course. Each point of the scale was described as follows:

1) Short definition and explanation of the topic.
2) Introductory coverage, such as typically given in basic studies.
3) Thorough coverage as in current textbooks.
4) Coverage updated with the latest conference and journal material.
5) Personal in-depth coverage based on instructor's (refereed) research.

Even the weight of 1 means a short coverage of the topic that is still more than just name-dropping, and weight 5 equals to an expert giving an in-depth coverage, e.g., a PhD-level person talking about his or her own field of research. The scale is not unambiguous, because of the overlap, but the idea is to choose the category that is the best match.

The result of this process is an abstraction of a course that is 'reverse-engineered' from the actual topics that are discussed in the lectures. The level of detail is such that it is possible to detect potential overlap between two courses. In addition, this abstraction can also be represented as a node-link diagram not unlike what a person might draw if requested to describe the contents of the course.

B. Collecting the curriculum data

The only method to collect the course outline at topic level is to request it directly from the lecturers. They can produce the outline on their own, but we believe that the best method is to have a meeting with the lecturer, along with handouts, and solve the arising concerns and problems collaboratively as the outline is constructed.

There are two primary problems in synthesising the curriculum by listing the topics and their weights: how to get the instructors to use, as much as possible, the same concepts to describe their topics, and how to maintain the same level of abstraction throughout the analysis. We used the following two-phase process to address these issues:

- *Initial outline from a 'tabula rasa' situation:* The instructor outlines the course and proposes the topics in an interview. If earlier related topics have been collected, the interviewer may propose a topic that is close or a synonym to the suggested one. We will present in Section 5 the tool we developed for this task.
- *Comparison against the full taxonomy of topics:* After the first round, the instructor sees the course outline as a diagram which incorporates all topics (but not the connections to other courses). The diagram is emailed to lecturers and they are given ample time to comment on the content.

The goal in this two-phase process was to encourage the instructors to stay at the same level of abstraction in their outlines, and to use the same labels for the same concepts whenever possible. In addition, the review round

allowed lecturers to see the full range of topics that had been mentioned, and allowed them to augment their outlines.

IV. Visualizing the curriculum

An indented list is perhaps the most familiar representation of an outline for most people, and if you ask a lecturer to visualize a course outline, you will probably get an indented list of topics in return. However, for the purpose of exploring overlaps, the indented list does not suffice. We need to have a representation where a single topic can be connected to multiple courses.

A. Visualizing curriculum data

Technically, our curriculum data is a node-link structure with two kinds of nodes: courses and topics, and weighted connections between them. A node-link structure is commonly visualized as a graph of nodes and connections. In Fig. 1, the curriculum structure is visualized as a graph or node-link diagram, a representation that is familiar to most people as well. Fig. 2 shows the type of node-link diagram we used initially: topics are stacked in the centre and courses can be either on the left or right side of this stack. This was a natural arrangement for our case, where courses from two institutes were compared. Weights of the connections are indicated by line thickness. The original idea was that courses with overlap would be placed vertically close to each other, and the overlapping topics would thus be easy to spot.

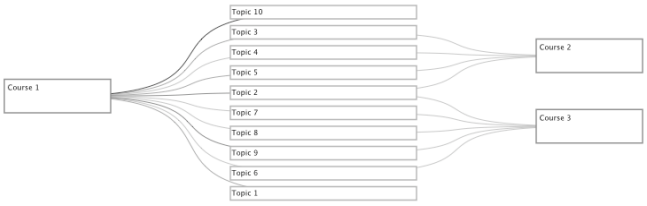

Figure 2. Initial design for a node-link diagram of courses and topics. In the current version, all the courses are on the left side of topics and the right side is used for the themes.

Initial, spontaneous user reactions to the layout in Fig. 2 made it clear that a one-level hierarchy is too restrictive. Users wanted to have a three-level hierarchy where the topics are combined into larger entities, or *themes*. This posed a problem, since the topics must be directly connected to courses – otherwise the smallest unit of overlap would be a theme. The solution was to use an inverted hierarchy, having courses on the left, topics in the centre, and themes on the right.

Another issue in the beginning was that the curriculum diagram soon grew larger than anticipated (over 1,800 elements). Initially, the diagram was produced with a generic drawing program, but soon the size and complexity of the diagram made this impractical. At that point, we begun to develop a dedicated computer program to process the diagrams.

The manual placement of diagram elements felt cumbersome in both solutions, so we implemented a fast heuristic to compute the element positions automatically, based on Sugiyama and Misue's algorithm [19]. As a heuristic, it does not guarantee an optimal placement of diagram elements, but in practice it manages to lower the number of link crossings sufficiently to make the diagram readable. This algorithm was originally developed to reduce edge crossings in digraphs, and it performs fast on our three-level, sideways-placed hierarchy.

Fig. 3 shows the complete curriculum visualization from our first real study, where the curriculums of two HCI units in Tampere, Finland were explored to improve the teaching cooperation. There are 27 courses, 483 topics, 27 themes, and 1356 connections in this diagram. When printed on paper, the diagram is ten A4 sheets in height.

B. Visualizing course distances

The automatic layout of the course diagram is based on *permutations*, or changing the order of data items without altering the information content. The underlying data structure for the diagram is a connection matrix which defines how the topics, courses, and themes are connected to each other. Permuting a connection matrix is the act of changing the order of rows and columns without affecting the connectivity information.

The layout heuristic permutes the positions of the courses, topics, and themes so that each element moves towards the vertical centre of elements it is connected to. This activity will reduce the crossings of connection lines, improve readability, and as a side effect, it will move (topically and thematically) similar courses closer to each other. However, it is possible that two courses end up together without significant similarity if one of the courses is connected to topics that in the layout get placed around the topics of the other course. Although this is rare, it may cause confusion to the users. To alleviate this problem, we constructed another view to visualize only the multidimensional distances between the courses (Fig. 4).

The distance matrix has courses in the columns and rows of the matrix, and the cell in the intersection visualizes the distance between the two courses. The grayscale value of the cell becomes darker as the distance between the courses becomes smaller, being black in the diagonal (i.e. a course has a zero distance to itself). The value is computed as a multi-dimensional Euclidean distance using both the intersecting and non-intersecting topics and their weights in the calculation. The actual distance calculation function is easy to replace, and we plan to experiment with other metrics in the future. The current metric was evaluated by users familiar with the curriculum who found it not to contradict their intuitive understanding of course contents.

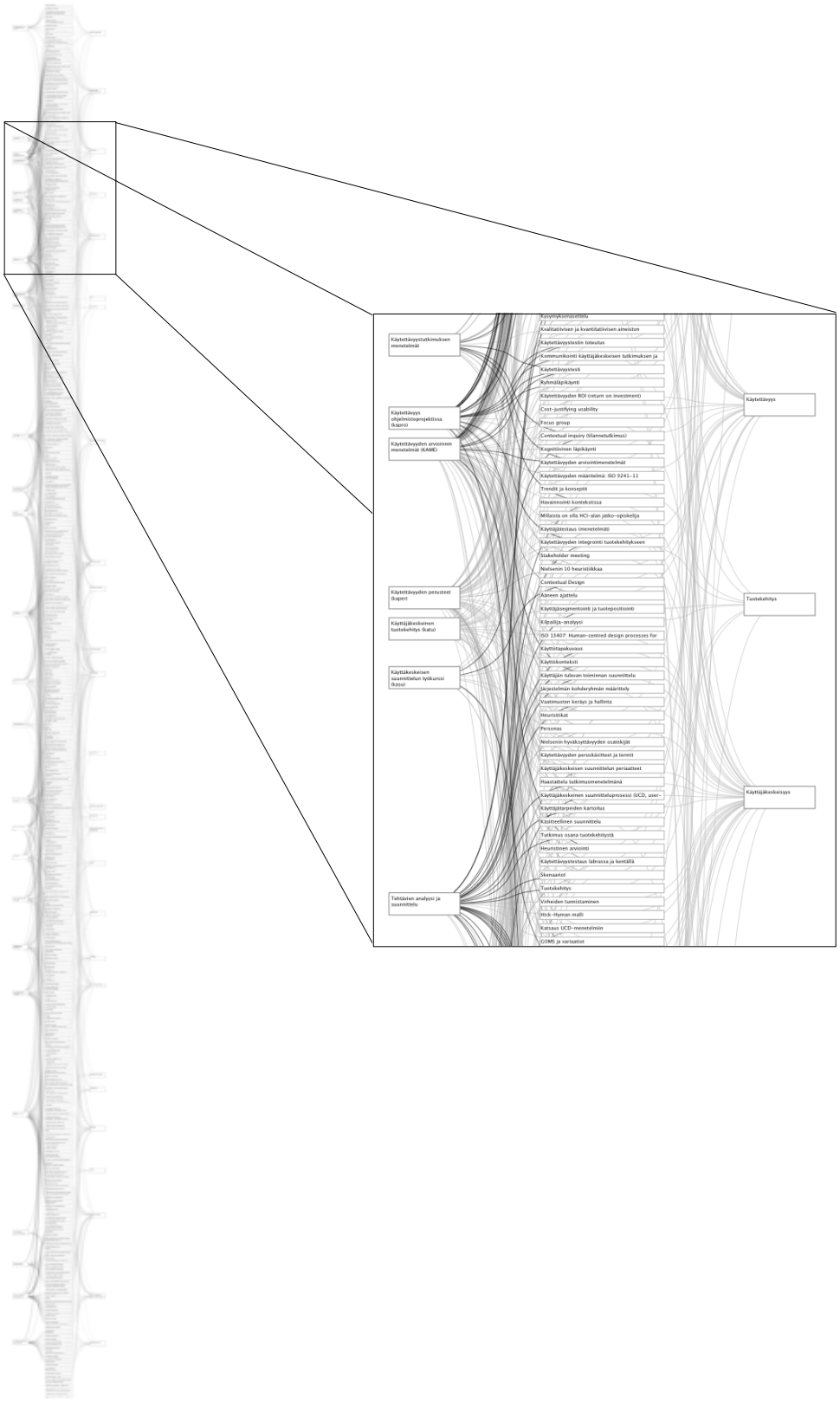

Figure 3. Complete diagram with 27 courses, 483 topics, 27 themes, and 1355 connections. A part of the diagram has been enlarged to show detail.

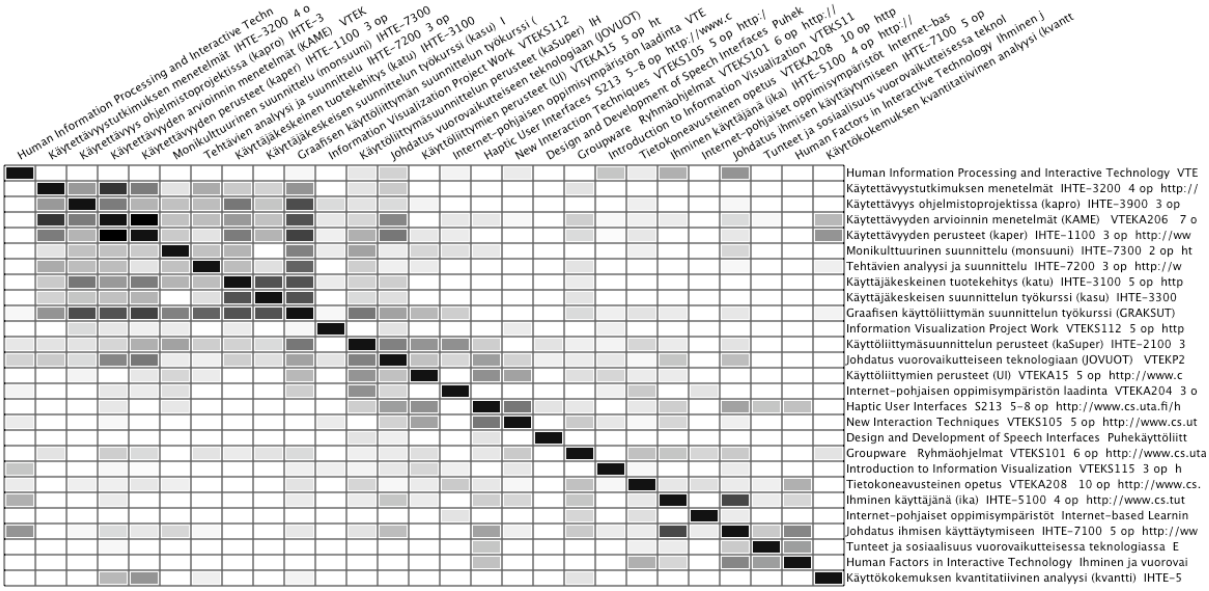

Figure 4. Distances between the 27 courses as grayscale values (darker the value, shorter the distance). Courses are listed both in the rows and in the columns, and the cell in the intersection of a row and a column gives the distance for the respective courses.

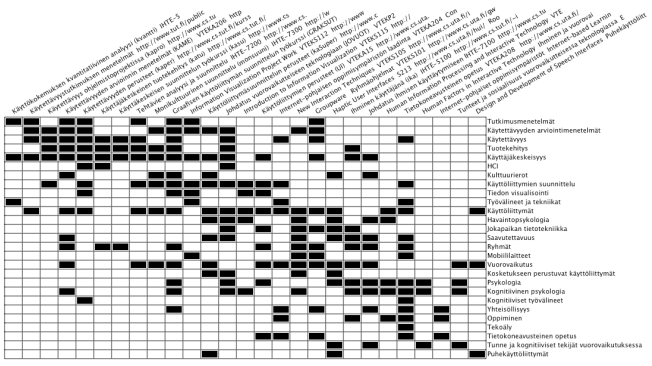

Figure 5. Thematic similarity between the courses as a binary matrix which is automatically rearranged to display similar courses next to each other. The courses are placed on the columns and the themes are listed on the rows. When the intersection of a row and a column is black, then the corresponding course includes at least some topics from that theme.

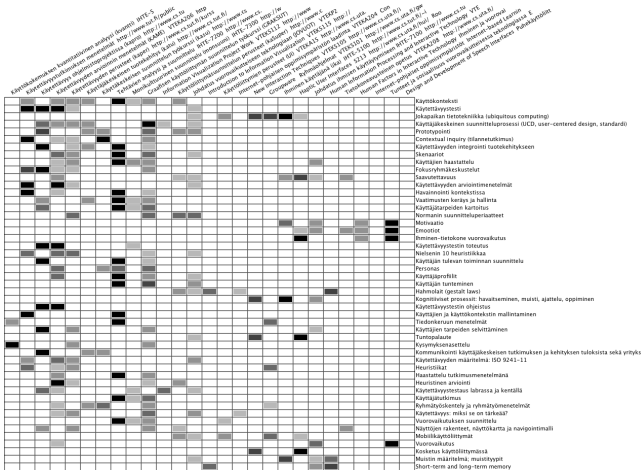

Figure 6. Cross tabulation of courses (in the columns) and topics (in the rows). The table is arranged descending according to sums of weights given to courses. (Only the top part of the table is illustrated.)

The matrix representation of distances inspired us to summarise other aspects of curriculum data in the same format. Fig. 5 shows the cross tabulation of 27 courses and 27 themes. This is a binary table – a course is either connected to a theme or not. The matrix is automatically reordered [20], [21] in such a way that thematically similar courses are placed next to each other. From this view, the user can quickly see which courses are potentially overlapping at this thematic level of detail.

A third view into the similarity of courses is the cross tabulation of courses and topics shown in Fig. 6. The courses are in the columns and the rows are arranged in descending order by the topic weights. The information content in this representation is the same as in Fig. 3, but in a tabular form.

The visualizations in figures 3–6 show different aspects of curriculum data and are meant to support different tasks in the curriculum analysis. Fig. 3 shows the collected data in full detail and facilitates analytical micro/macro reading as suggested by [22], [23]: ‘detail cumulate into larger coherent structures’, providing an overview of the whole data. The distance view in Fig. 4 quickly reveals if there are any obvious groups in the data (such as the darker rectangle close to the upper left corner of the matrix: almost all of these topically tightly knit courses are from the same unit, reflecting their deep coverage of design and evaluation of user experience topics). Fig. 5 displays again

an automatically rearranged matrix to show which courses are similar in covering larger themes of the curriculum. Although the assignment of topics into themes is a subjective labelling, the representation reveals which courses cover the spefic themes. Finally, Fig. 6 displays the data in an order which reveals the overall weight given to each topic in the curriculum.

V. Interacting with the Visualization

Although the visual representations of the previous section are not particularly complex or large, there are tasks like finding a certain data item or comparing two sets of topics that are clearly laborious and error-prone to perform manually. This section describes a software application that was developed to process and visualize curriculum data.

A. Main interface

Fig. 7 shows the main view of the application that was developed to interact with the curriculum visualizations. The diagram view is a conventional direct manipulation editor for the diagram, allowing the user to create, delete, and connect diagram elements. Selecting a single diagram element shows its data in the text view where it can be edited. The first few lines of this text are used as a label, and the rest can be used, for example, as descriptions or notes regarding the element. The text view recognises URL's, allowing the creation of shortcuts to the course descriptions on the web.

One of the most common operations with the diagram editor is to look for an element that has a certain string in its name or description. There is a text field for the search string and a button for activating the search. The search field displays the number of found occurrences next to the search button, and it allows a circular traversal of matching elements. The main view will automatically scroll according to the current match.

The weight of a topic-course connection is visually encoded in the width of the connection line. As seen in Fig. 3, this encoding is too weak when the number of connections increases and the view becomes cluttered. We duplicate this encoding when the user selects a course by highlighting the connected topics with an overlaid bar. The bar has a length that represents the weight of the connection, extending the whole box when the weight is five and one fifth from the left when the weight is one, as seen in Fig. 7.

There is interaction between the main view and the three matrix views (Figs. 4-6) that were discussed in the previous section. If there are no selections in the main view, the matrix views will summarise all the courses, but otherwise the matrix views will summarise only the courses selected in the main view (Fig. 3).

B. Connection dialog

Connection management by direct manipulation is intuitive for small diagrams, but with a larger diagram it is awkward and error-prone to make a connection between elements that are far apart, although the diagram view autoscrolls during drag operations. We implemented a dialog with a dynamic query interface [24] to make connection management easier (Fig. 8).

The connection dialog is always related to a course or a theme from which it is opened. The dialog displays all the topics with a slider to manipulate the corresponding weight. Topic selection is implemented as a dynamic query. As soon as the user starts to type, the dialog will display the first topic that matches the typed letters (and highlights the rest). If the matching topic is not yet connected to the current element, the button will show 'Connect', and 'Disconnect' otherwise. If there is no match, the button will show 'Add topic'.

The dynamic query interface was implemented to support a situation where a researcher interviews a lecturer face-to-face and needs to rapidly suggest topics that might be viable synonyms to the one offered by the lecturer. This is crucial to maintaining the cohesion of the data, since the overlap analysis will deteriorate if there are a lot of synonyms in the diagram.

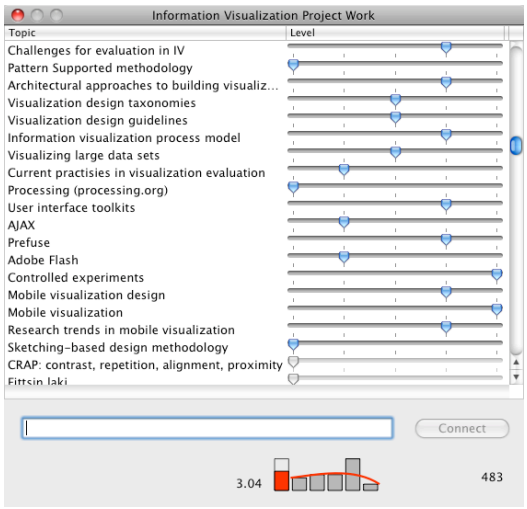

Figure 8. Connection management dialog for the course 'Information Visualization Project Work'. The user can rapidly explore the matching topics by giving a substring of topic name, and then manage the connection state and its weight. The display at the bottom of the dialog shows the average of the given weights (3.04, the red bar) and distribution of weights (bars represent values from one to five, from left to right).

The bar chart at the bottom of the dialog shows the distribution of weights for the current course or theme at the moment. The first red bar shows the mean of the weights (shown as a number as well), and the grey bars show the relative share of weights from one to five, respectively. The same bar chart display is shown in the diagram view as well when a single course or theme is selected. The distribution view is an approximate indicator for the level of a course, and in our experience it works quite well (advanced studies average over three, basic studies less than two), and

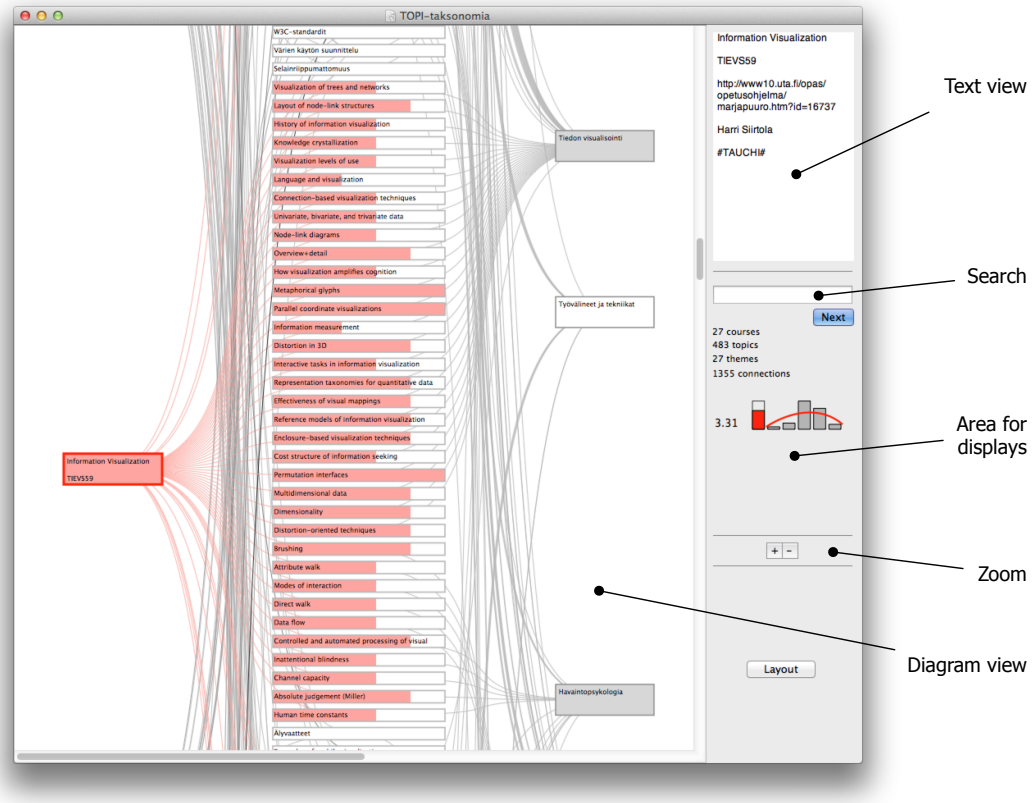

Figure 7. Main view of the user interface. In the *Diagram view* the user can select and connect diagram elements by direct manipulation. The panel on the right displays details about the currently selected diagram element (*Text view*), allows to search elements having a certain character string in their name or description (*Search*), and displays details about the whole diagram and the currently selected elements.

communicates how the lecturer sees the comprehensiveness of the course material.

C. Overlap display

The distance view (Fig. 4) shows if two courses have a short distance, that is, have overlap, but it does not indicate where the overlap originates from. The diagram view allows one to drill down into the overlap when two courses are selected. As seen in Fig. 9, the overlapping topics have weight bars for both courses, and the user is able to inspect which topics are actually overlapping.

In addition to the main view, the overlap is summarised in the display area with a visualization that contains two percentages. The areas of the rectangles in the visualization represent the sum of all topics for corresponding courses, and the percentages indicate how much the overlapping part is from each of the two courses.

VI. Discussion

This work was carried out as part of the incipient teaching cooperation of two HCI units in the Tampere region, IHTE and TAUCHI, between autumn 2007 and autumn 2008. One of our first aims was to analyse and report the potential overlap in the programs offered by the two units, and to plan how we could cooperate more efficiently. We were unable to find a satisfactory approach to analyse the curriculums, so we decided to develop our own.

In the early stages of this survey there were concerns regarding the extent to which we can disrupt the life of the teaching staff with the outlining requests. Some of them were reluctant to produce detailed outlines, but the majority humoured us, and some were even keen to see the results. Two workshops were organised to disseminate and discuss the findings.

In the initial stage the analysis produced 27 themes. The steering board guiding the analysis cut the number of themes into 6 to make it easier to connect topics and themes without ambiguity. In the final version, the average number of course-theme connections was 4 (min. 1, median 4, mean 3.7, max. 6). The average number of topics for a course given in the outline was 26 (min. 8, median 26, mean 32.1, max. 78).

The fundamental finding of the analysis was that although the two units both operate in the field of human-computer interaction, they emphasise different sub-areas in their cur-

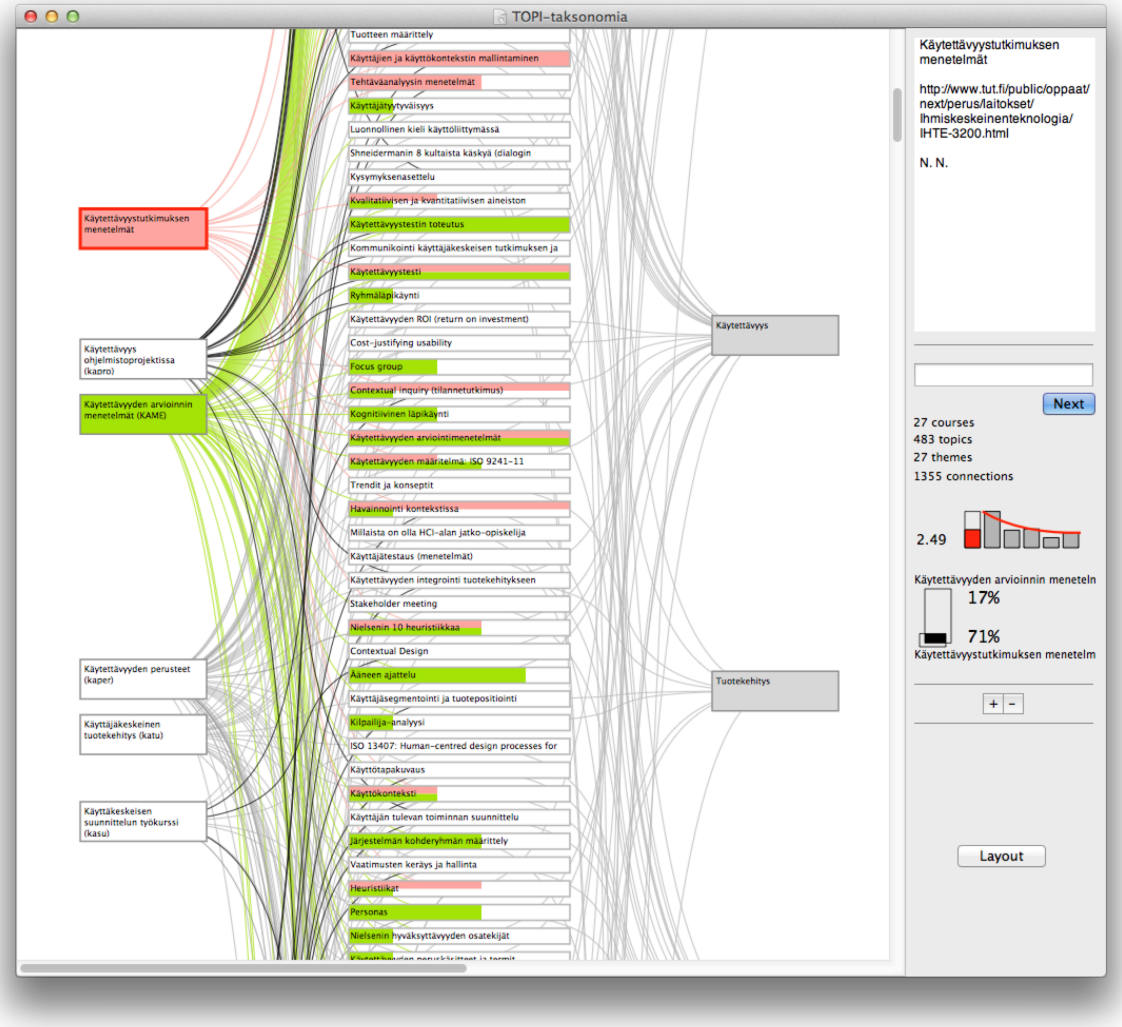

Figure 9. Visualizing the overlap of two courses. The courses ‘Käytettävyystutkimuksen menetelmät’ (‘usability research methods’) and ‘Käytettävyyden arvioinnin menetelmät’ (‘usability evaluation methods’) have been selected and their topics and themes are highlighted. The display on the right gives the combined distribution of these two courses, and shows the proportion of the overlap against all the topics in a course (71 and 17 percent, respectively).

riculums (which can be seen in their research programs as well). This is apparent in Fig. 4: there is a darker rectangle on the upper left part of the matrix. Those tightly connected courses are almost all from one unit, with the exception of two courses. This both revealed the difference in the focus, and pointed out where the potential for a closer cooperation is.

Our survey did reveal some intra-unit and inter-unit overlap that was acted upon. Some of those issues were not surprising, but there were also interesting revelations in the details. One of them was that the gestalt laws of visual design appeared in surprisingly many courses!

The current software application was developed for the needs of our survey, without plans for further development. Now that we have made our curriculums visible in this manner, it seems worthwhile to update the course descriptions periodically, and to use the application in curriculum planning and quality control (as part of a ‘curriculum management system’). The current application stores the course data as XML descriptions, and we have plans to implement a web application that would allow the lecturers to work on the course outlines on their own. After all, the quality of results depends totally on the accuracy of the outlines.

VII. Conclusions

We have presented a new kind of method to visualize and analyse overlap in curriculums. The method is based on a bottom-up synthesis of curriculum data using sets of courses, topics, themes, and weighted connections between them. We constructed a software application to support the

process and proposed several different views for interacting with the visualizations. Our tool proved to be a success in the first study and two other studies are currently underway.

ACKNOWLEDGEMENTS

We acknowledge the useful feedback provided by the steering board of TOPI project ('Tampere region research and teaching cooperation in humane technology'): Kaisa Väänänen-Vainio-Mattila, Sari Kujala, Roope Raisamo, and Teija Vainio. We also thank Tomi Heimonen and Saila Ovaska for their useful comments.

This research was financially supported by the University of Tampere (project number 76423) and the Finnish Ministry of Education.

REFERENCES

[1] R. Spence, *Information Visualization – Design for Interaction*. Prentice-Hall Europe, Pearson Education Ltd., 2007.

[2] H. Siirtola, "Interactive visualization of multidimensional data," Dissertations in Interactive Technology, Number 7, University of Tampere, 2007. [Online]. Available: http://acta.uta.fi/english/teos.php?id=10949

[3] F. W. English, "Curriculum mapping," *Educational Leadership*, vol. 37, no. 7, pp. 558–559, 1980.

[4] I. Hege, D. Nowak, S. Kolb, M. R. Fischer, and K. Radon, "Developing and analysing a curriculum map in occupational – and environmental medicine," *BMC Medical Education*, vol. 10, p. 60, 2010. [Online]. Available: http://www.biomedsearch.com/nih/Developing-analysing-curriculum-map-in/20840737.html

[5] S. K. Card, J. D. Mackinlay, and B. Shneiderman, *Readings in Information Visualization: Using Vision to Think*. Morgan Kaufmann Publishers, 1999.

[6] M. L. Davison, *Multidimensional Scaling*. John Wiley & Sons, 1983.

[7] D. W. Johnson, F. Wilkes, P. Ormond, and R. Figeuroa, "Adding value to the IS'97... curriculum models: An interactive visualization and analysis prototype," *Journal of Information Systems Education*, vol. 13, no. 2, pp. 135–142, 2002.

[8] J. Sant, "Visualization of curricula using multi-dimensional scaling," in *Proceedings of World Conference on Educational Multimedia, Hypermedia and Telecommunications 2004*, L. Cantoni and C. McLoughlin, Eds., 2004, pp. 1911–1917.

[9] P. Gestwicki, "Work in progress - curriculum visualization," 38th ASEE/IEEE Frontiers in Education Conference, Session T3E, 2008.

[10] W. F. Clocksin and C. S. Mellish, *Programming in Prolog: Using the ISO Standard*. Springer, September 2003.

[11] L. Sommaruga and N. Catenazzi, "Curriculum visualization in 3D," in *Web3D '07: Proceedings of the twelfth international conference on 3D web technology*. ACM, 2007, pp. 177–180.

[12] E. Prasolova-Førland, "Creative curriculum visualization in a 3D CVE," in *Proc. 9th IASTED International Conference on Computers and Advanced Technology in Education (CATE 2006)*. ACTA Press, 2006, pp. 404–410.

[13] A. Katifori, C. Halatsis, G. Lepouras, C. Vassilakis, and E. Giannopoulou, "Ontology visualization methods—a survey," *ACM Comput. Surv.*, vol. 39, no. 4, p. 10, 2007.

[14] A. Katifori, E. Torou, C. Halatsis, G. Lepouras, and C. Vassilakis, "A comparative study of four ontology visualization techniques in Protégé: Experiment setup and preliminary results," in *Proceedings of International Conference on Information Visualisation (IV'06)*. IEEE Computer Society, 2006, pp. 417–423.

[15] A. J. Pretorius, "Lexon visualization: Visualizing binary fact types in ontology bases," in *IV '04: Proceedings of the Information Visualisation, Eighth International Conference*. IEEE Computer Society, 2004, pp. 58–63.

[16] L. N. Cassel, G. Davies, W. Fone, A. Hacquebard, J. Impagliazzo, R. LeBlanc, J. C. Little, A. McGettrick, and M. Pedrona, "The computing ontology: application in education," in *ITiCSE-WGR '07: Working group reports on ITiCSE on Innovation and technology in computer science education*. ACM, 2007, pp. 171–183.

[17] S. Kriglstein, "Analysis of ontology visualization techniques for modular curricula," in *USAB*, ser. Lecture Notes in Computer Science, A. Holzinger, Ed., vol. 5298. Springer, 2008, pp. 299–312.

[18] A. Holzinger, Ed., *HCI and Usability for Education and Work, 4th Symposium of the Workgroup Human-Computer Interaction and Usability Engineering of the Austrian Computer Society, USAB 2008, Graz, Austria, November 20-21, 2008. Proceedings*, ser. Lecture Notes in Computer Science, vol. 5298. Springer, 2008.

[19] K. Sugiyama and K. Misue, "Visualization of structural information: Automatic drawing of compound digraphs," *IEEE Transactions on Systems, Man and Cybernetics*, vol. 24, no. 4, pp. 876–892, 1991.

[20] H. Siirtola and E. Mäkinen, "Constructing and reconstructing the reorderable matrix," *Information Visualization*, vol. 4, no. 1, pp. 32–48, 2005. [Online]. Available: http://dx.doi.org/10.1057/palgrave.ivs.9500086

[21] E. Mäkinen and H. Siirtola, "Reordering the reorderable matrix as an algorithmic problem," in *Diagrams 2000: Proceedings of the First International Conference on Theory and Application of Diagrams*. Lecture Notes in Artificial Intelligence 1889, Springer-Verlag, 2000, pp. 453–467.

[22] E. R. Tufte, *The Visual Display of Quantitative Information*. Graphics Press, 1983.

[23] ——, *Envisioning Information*. Graphics Press, 1990.

[24] B. Shneiderman, "Dynamic queries for visual information-seeking," *IEEE Software*, vol. 11, no. 6, pp. 70–77, 1994.